



UNIVERSIDADE
FUMEC

Estrutura de Dados

```
1 // Estrutura de Dados - Funções de Escopo Local  
2 #include <stdio.h>  
3  
4 // Função principal  
5 int main()  
6 {  
7     int valor;  
8     printf ("Digite um valor para a função main: ");  
9     scanf ("%d", &valor);  
10    printf ("Na função main - %d ", valor);  
11  
12    // Função de Escopo Local  
13    escopoLocal ();  
14  
15    return 0;  
16 }  
17  
18 // Função de Escopo Local  
19 void escopoLocal ()  
20 {  
21     int valor;  
22     printf ("Digite um valor para a função escopoLocal: ");  
23     scanf ("%d", &valor);  
24     printf ("Na função escopoLocal - %d ", valor);  
25 }
```

VARIÁVEIS

APRESENTAÇÃO

Prezado(a) aluno(a), neste módulo conversaremos sobre a utilização das variáveis em seus códigos, você saberá quando e como utilizá-las e poderá diferenciar seus diversos tipos e formas de aplicação.

Compreenderá quando e como utilizar um vetor em seus programas e quais as características relevantes de cada tipo de variável.

OBJETIVOS DE APRENDIZAGEM

Ao final desse módulo você deverá ser capaz de:

- Compreender as características de uma variável;
- Identificar as formas de utilização de cada variável;
- Diferenciar variáveis comuns e compostas.

VARIÁVEIS

Introdução



Você já parou para pensar como é feito o armazenamento de dados em um programa? Afinal, os itens que digitamos, ou que carregamos para um programa, são armazenados de que forma? Por quem? Como?

A partir de agora compreenderemos a primeira etapa deste tipo de utilização, ou seja, como a memória é utilizada.

A memória física de um computador é utilizada para efetuar os armazenamentos que ele necessita em diversos programas e dispositivos. Quando, em um programa, precisamos armazenar uma informação específica, na verdade “separamos” um espaço desta memória para isto, a este espaço damos o nome de variável. Vejamos alguns exemplos!

- Se precisar armazenar o nome de alguém, criará uma variável para guardá-lo;
- Se precisar armazenar a idade, criará uma variável para guardá-la;
- E isto deverá ocorrer para TODAS as informações que você deseja armazenar em seu programa.

Bem, então, esse espaço de memória nos permite guardar informações para utilizarmos a qualquer momento no programa.

Mas, para que essa utilização aconteça você precisa ficar atento a algumas características relevantes.

Vamos então descobrir que características são essas!



Variável

- 1 • Nome
- 2 • Tipo de dado
- 3 • Valor armazenado (conteúdo)
- 4 • Endereço de memória
- 5 • Escopo
- 6 • Tempo de vida

Agora detalharemos cada uma dessas características.

Nome de uma variável

Ao indicarmos um nome a uma variável, estamos a identificando dentro do programa.

ATENÇÃO

O nome de uma variável deve seguir regras, para que o programa funcione corretamente. É importante observar que algumas regras fazem parte da sintaxe da linguagem, e outras são convenções, que no mercado de trabalho dão agilidade à escrita e manutenção dos programas.



Ambas devem ser respeitadas e seguidas! Lembre-se que você não escreve um programa apenas para você ler, outras pessoas utilizarão e deverão compreender seu código.

Ao definir o nome você precisa:

- **Começar com letras ou underline (_):** Sempre que criar uma variável é imprescindível que ela comece com letras ou underline (_), assim ela será aceita pelo programa. A partir da segunda posição pode-se utilizar letra, número ou underline (_).

Exemplo: idade, nome1200, salario_final, somaTotal

- **Utilizar letras minúsculas:** As variáveis são escritas, preferencialmente, com letras minúsculas. Se houver duas ou mais palavras, cada nova palavra deverá iniciar com letra maiúscula (este item é conhecido como Camel Case).

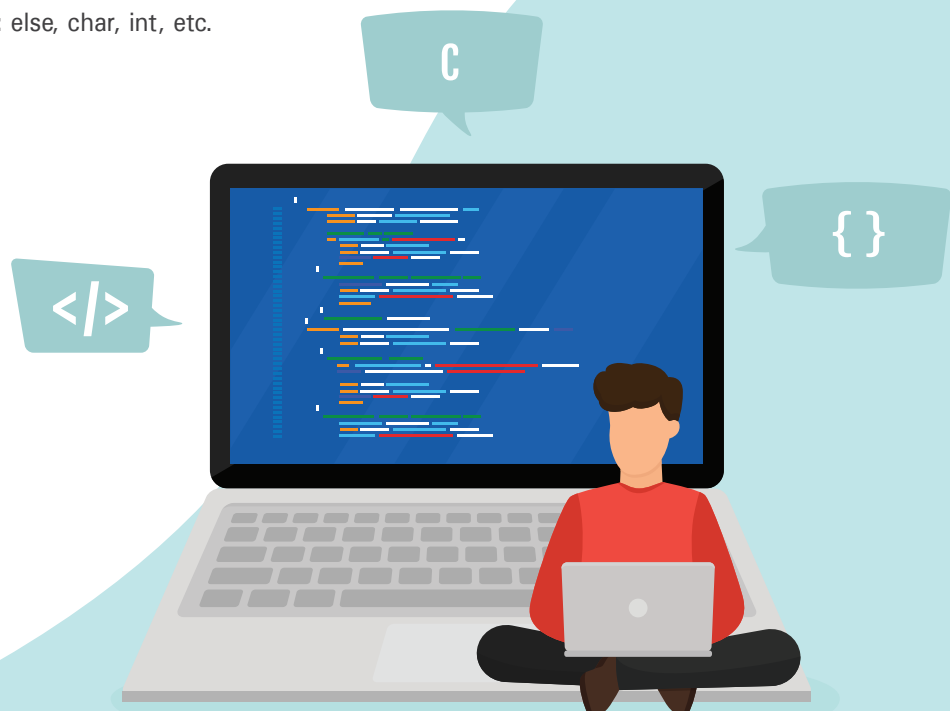
Exemplo: maiorIdade, salarioFinal, media_geral, salario, idade2

- **Nomes simples e descritivos:** É importante que o nome de uma variável seja de fácil compreensão para você, que está escrevendo o programa, e para as pessoas que darão manutenção no código.

Exemplo: uma variável que possui o nome "idade" armazenará a idade de algo, ou alguém, dentro do código. É fácil concluir isto.

- **Não utilize palavras reservadas:** Algumas palavras pertencem a estrutura da linguagem, e não podem ser utilizadas para nomes de variáveis.

Exemplo: else, char, int, etc.



Tipo de uma variável

Definir o tipo de uma variável é o mesmo que definir o que ela poderá armazenar, ou seja, qual o conteúdo você poderá guardar dentro daquele espaço de memória.

Você pode utilizar três classificações possíveis de variáveis!

Sua variável poderá ser numérica, e neste caso, você poderá armazenar valores reais ou inteiros dentro dela, declarando-a do tipo int, float ou double.

Ou, se necessário, sua variável poderá ser do tipo alfanumérica, e você armazenará um caractere dentro dela, declarando-a com o tipo char.

E você já ouviu falar em modificadores de tipos de variáveis? Não!?

Os tipos citados são conhecidos como básicos, ou primitivos, além deles, também podemos utilizar modificadores para variar a declaração, essa utilização visa fazer o armazenamento de memória ideal.

Para esclarecer melhor vamos a um exemplo...



Uma variável inteira ocupa 4 bytes na memória, mas pode ser que, no seu código, não será necessário gastar todos esses bytes, pois a sua utilização será sempre bem menor que isto, portanto, o ideal é ter um tipo disponível que ocupe menos espaço de memória. Na linguagem C, quem é responsável por fazer este “ajuste” são os modificadores de tipo.

Um modificador efetua uma alteração no tipo original da variável, permitindo que seu armazenamento de memória seja maior ou menor, dependendo de sua aplicação.

Os modificadores possíveis são:

- **Short:** indica que a variável será menor que seu valor original.
Por exemplo, uma variável inteira armazena 4 bytes, se ela for short int armazenará 2 bytes.
- **Long:** indica que a variável será maior que seu valor original.
Observação: Para o long, é necessário compreender o tipo de arquitetura em que está trabalhando em seu computador, pode ser que ele utilize a mesma quantidade de bytes ou mais.
- **Unsigned:** indica que a variável não armazenará valores negativos.
Por exemplo, uma variável short int armazena valores de -32.765 a 32.767, se ela for unsigned short int armazenará de 0 até 65.535.

IMPORTANTE

Observações importantes para esta utilização:

- Toda variável é naturalmente do tipo signed, pois sempre permite valores negativos e positivos;
- Ao tipo float não se pode aplicar nenhum modificador;
- Ao tipo double pode-se aplicar apenas o modificador long;
- Todos os modificadores podem ser aplicados ao tipo int.



Cada compilador é livre para escolher tamanhos adequados para o seu próprio hardware, com algumas restrições:

- Os short's e int's devem ocupar pelo menos 16 bits;
- Os longs pelo menos 32 bits;
- O short não pode ser maior que int;
- O int não pode ser maior que long.

Veja um quadro com os tipos possíveis para declaração, seu armazenamento na memória (bytes) e suas possibilidades de receber conteúdo.

TIPO		NÚMERO DE BYTES	INICIO	FIM
Caractere	char	1	-128	127
	unsigned char	1	0	255
Número inteiro	short int	2	-32.765	32.767
	unsigned short int	2	0	65.535
	int	4	-2.147.483.648	2.147.483.647
	unsigned int	4	0	4.294.967.295
	long int	4	-2.147.483.648	2.147.483.647
	unsigned long int	4	0	4.294.967.295
Número real	float	4	3.4×10^{-38}	3.4×10^38
	double	8	1.7×10^{-308}	1.7×10^{308}
	long double	8	3.4×10^{-4932}	3.4×10^{4932}
void	void	0	Nenhum valor	



Agora que você sabe que cada tipo possui um armazenamento, em bytes, na memória. Já pensou em como descobrir isso? Afinal, seria quase impossível precisarmos decorar. Concorde?

Para facilitar nossa vida, e descobrirmos a quantidade de bytes que cada tipo utilize, aplicaremos o comando `sizeof` da linguagem, veja um exemplo no código:

```

1  #include <stdio.h>
2  int main () {
3      printf ("\nO armazenamento de uma variável do tipo "
4              " int é %i bytes ", sizeof(int));
5      return 0;
6  }
7

```

Neste código, o comando irá exibir quantos bytes o tipo `int` gasta na memória.

Escopo

Definir o escopo de uma variável, indica saber onde ela será válida, ou seja, onde ela será visível para a execução do programa, neste caso, onde poderá ser utilizada.

Existem duas formas de declararmos, e definirmos a visibilidade, de uma variável, ela pode ser global ou local.

Escopo - Global

Uma variável é considerada global quando declarada fora de todas as funções. Neste caso, ela será visível por todo o programa, ou seja, ela não “pertence” a nenhuma função específica, qualquer uma poderá acessar, e alterar, seu conteúdo.

```
1  #include <stdio.h>
2  int valor;
3
4  void soma() {
5      printf("\n%d ", valor + valor);
6  }
7
8  int main () {
9      printf("\nDigite um valor: ");
10     scanf("%d", &valor);
11     soma();
12     return 0;
13 }
14
```

Para o código anterior, observe que a variável “valor” não foi declarada dentro de nenhuma função, portanto, a função soma e a função main poderão utilizar e alterar seu conteúdo, pois ela é global.

Escopo - Local

Uma variável é considerada local quando declarada dentro de uma função, e só poderá ser visível, e alterada, pela função em que foi declarada.

ATENÇÃO

Os parâmetros de uma função são considerados variáveis locais, pois possuem as mesmas características.



```
1  #include <stdio.h>
2  void escopoLocal();
3
4  int main() {
5      int valor;
6      printf ("\nDigite um valor para a função main: ");
7      scanf("%d", &valor);
8      escopoLocal();
9      printf ("\nNa função main - %d ", valor);
10     return 0;
11 }
12
13 void escopoLocal() {
14     int valor;
15     printf ("\nDigite um valor para a função escopoLocal: ");
16     scanf("%d", &valor);
17     printf ("\nNa função escopoLocal - %d ", valor);
18 }
19
```

No código anterior, as variáveis `valor`, apesar de terem o mesmo nome, são variáveis distintas, ou seja, ocupam espaços de memória **diferentes**.

As regras que regem onde uma variável é válida são:

- Duas variáveis globais não podem ter o mesmo nome.
- O mesmo vale para duas variáveis locais, de uma mesma função.
- Duas variáveis locais, de funções diferentes, podem ter o mesmo nome sem perigo de conflito. Mas atenção, não serão a mesma variável.

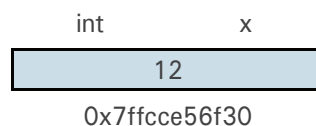


Endereço de memória

Mas, afinal, após verificar todas essas características você ainda deve estar pensando, “Mas como essas variáveis ficam alocadas na memória?” É uma ótima pergunta.



E para respondê-la você precisará compreender uma característica muito importante, o endereço de memória. Ele é responsável por identificar uma variável na memória, então, saiba que cada variável tem seu endereço, na verdade, cada byte da memória tem um endereço, portanto, variáveis que utilizam mais de um byte apenas indicam o endereço do primeiro byte (mesmo que estejam utilizando mais de um).



Na variável x, que possui o conteúdo 12, o endereço de memória é 0x7ffcce56f30c.

DECLARAÇÃO

Declarar variável é o mesmo que “separar” um espaço de memória para armazenar os dados que serão úteis aos programas.

Para declararmos uma variável, em C, basta utilizarmos a seguinte sintaxe:

```
tipo_da_variável nome_da_variável;
```

Onde:

O tipo da variável, segue o quadro de tipos possíveis, que já conhecemos juntos nas páginas anteriores e o nome da variável segue as descrições já citadas aqui.

Veja a seguir alguns exemplos destas declarações:

```
1 int main () {  
2     int idade;  
3     float somaTotal, total;  
4     double salario;  
5     char letra, frase[30];  
6     return 0;  
7 }  
8
```

Seguindo os exemplos acima, podemos utilizar qualquer tipo, e modificadores, de variável para declaração.

Se as variáveis forem do mesmo tipo, pode-se declará-las apenas separando por vírgulas e, ao finalizar, incluir ponto e vírgula.

Atribuir valor

Atribuir valor a uma variável é o mesmo que colocar um valor dentro do espaço de memória declarado para ela, e, neste caso, deve respeitar os valores permitidos para cada tipo de variável. O operador responsável por fazer isto é o = (igual).

Veja o exemplo de utilização deste operador:

```
idade = 45  
letra = 's'  
somaTotal = 1234.89  
pesoAtual = 62
```

INICIALIZAÇÃO

Inicializar uma variável é o mesmo que dar um valor para que ela comece o programa, ou seja, incluir um conteúdo nesta variável.

Considerando as necessidades do programa, uma variável deve ser inicializada a partir de duas regras importantes:

IMPORTANTE

1ª Regra: Toda variável que recebe ela mesma precisa ter um valor inicial para começar a lógica.

2ª Regra: Toda variável que for testada, ou utilizada, antes de receber um valor precisa ser inicializada para que a ação tenha resultado.



Nos demais casos, você pode inserir valor a uma variável sempre que precisar, não existe regra específica. Para isto, utilize o operador de atribuição.

Uma variável pode ser inicializada em dois momentos do programa:

1. Ao declará-la -

```
int idade = 12;
```
2. Durante as linhas de código, obviamente, após a declaração -

```
int idade;  
  
idade = 12;
```

ATENÇÃO

Quanto a utilização, fique atento:

- Os valores reais devem ser escritos com ponto;
- Os valores atribuídos sempre devem atender ao tipo da variável, ou seja, não se deve colocar um conteúdo alfanumérico dentro de uma variável numérica, etc;
- O operador de atribuição (=) é diferente do operador relacional de igualdade (==);
- Variáveis do tipo char armazenam seus conteúdos com aspas simples (').



Simplificando expressões

Em códigos de programação, é comum que uma variável precise armazenar seu próprio conteúdo, neste caso, dizemos que a variável recebe ela mesma, ou seja, mantém seu próprio valor para efetuar o cálculo.

Toda vez que uma variável precisar receber ela mesma, uma forma prática, e muito utilizada, para simplificar as expressões, é utilizar o operador de atribuição da seguinte forma:

<pre>float somaIdade = 0, idade = 26; //As inicializações são apenas para exemplificar</pre>	
Código comum	Código simplificado
<pre>somaIdade = somaIdade + idade;</pre>	<pre>somaIdade += idade;</pre>
<pre>somaIdade = somaIdade * 10;</pre>	<pre>somaIdade *= 10;</pre>
<pre>somaIdade = somaIdade / 150;</pre>	<pre>somaIdade /= 150;</pre>
<pre>somaIdade = somaIdade % 5;</pre>	<pre>somaIdade %= 5;</pre>

Vetores e Matrizes

Bem, até agora conversamos sobre variáveis comuns, que ocupam apenas um espaço de memória, a partir de agora falaremos sobre variáveis compostas, que ocupam dois ou mais espaços de memória!



Uma variável composta corresponde à declaração de vetores e matrizes para utilização em um código.

Estes tipos de variáveis só são utilizadas quando precisamos armazenar vários valores do mesmo tipo, evitando assim a declaração de inúmeras variáveis.

Todas as características citadas anteriormente são utilizadas para variáveis compostas, declaração de nomes, inicialização, escopo, etc, mas, além do nome, elas também são identificadas com um índice e, uma característica que é importante salientar, é o endereço de memória, em vetores e matrizes, este endereço torna-se sequencial.

VARIÁVEIS COMPOSTAS UNIDIMENSIONAIS: VETORES

Também são conhecidas como vetores, e possuem apenas uma dimensão.

A sintaxe para declarar um vetor é:

```
tipo nome_do_vetor[tamanho];
```

Onde:

- O tipo segue a lista de variáveis comuns;
- O nome segue as regras de variáveis comuns;
- O tamanho representa a quantidade de elementos que o vetor irá conter.

IMPORTANTE

É importante saber que esses elementos são indexados a partir do valor 0 (zero).



A inicialização de um vetor pode ser definida ao declará-la, bastando indicar os elementos entre chaves, separados por vírgula.

Veja o exemplo a seguir:

```
int vetorExemplo[3] = {12, 45, 83};
```

int vetorExemplo		
0	12	0x7ffcb0638150
1	45	0x7ffcb0638154
2	83	0x7ffcb0638158

OBSERVAÇÃO

Observação importante: Na arquitetura que estou utilizando, meu tipo inteiro utiliza 4 bytes para cada armazenamento, por isso que o cálculo “pula” de 150 para 154, e depois para 158, ou seja, de 4 em 4 bytes ele faz o armazenamento de uma variável inteira.



VARIÁVEIS COMPOSTAS BIDIMENSIONAIS: MATRIZES

Possuem duas dimensões, definindo linha e coluna para preenchimento. Também conhecido como matriz.

A sintaxe para declarar uma matriz é:

```
tipo nome_da_matriz[linha][coluna];
```

Onde:

- O tipo segue a lista de variáveis comuns;
- O nome segue as regras de variáveis comuns;
- A linha representa a quantidade de linhas que a matriz irá conter.
- A coluna representa a quantidade de colunas que a matriz irá conter.



A inicialização de uma matriz pode ser definida ao declará-la, bastando indicar os elementos entre chaves, separados por vírgula.

Veja o exemplo a seguir:

```
int matrizExemplo[2][3] = {0,1,2,3,4,5};
```

	0	1	2
0	0x7ffee13fe610	0x7ffee13fe614	0x7ffee13fe618
1	0x7ffee13fe61c	0x7ffee13fe620	0x7ffee13fe624

VARIÁVEIS COMPOSTAS NÃO DIMENSIONADAS

Não dimensionar um vetor, ou matriz, significa não indicar o tamanho que ela terá, neste caso, apenas a inicialização definirá o valor.

Veja os exemplos a seguir:

```
double valores[] = {34.6, 23, 9.7, 12}; //o vetor possui tamanho 4.
```

```
int numeros[][2] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }; //A matriz possui tamanho 5x2
```

ATENÇÃO

Para uma matriz, só é possível deixar um valor sem dimensionamento, para a linguagem C, é importante indicar a quantidade de colunas, mesmo inicializando.



Síntese

Bem, terminamos este módulo por aqui. Nele você pode perceber o que são variáveis e como são declaradas e localizadas na memória.

Descobriu que variáveis são espaços de memória declaradas para armazenarmos dados e utilizarmos durante todo o programa.

Percebeu que elas possuem características importantes, que são: o nome, que identifica uma variável no programa; o tipo, que define o que será inserido dentro do espaço de memória criado; conteúdo, que corresponde ao que foi atribuído ao espaço de memória; escopo, que é o local onde a variável está visível, ou disponível, para ser utilizada; e, por fim, o endereço de memória, que a identifica e permite localizá-la na memória alocada.

Agora você também sabe que podemos utilizar variáveis compostas, que podem ter uma ou mais dimensões, que carrega todas as características citadas e possui seus endereços sequenciais na memória.

Bem, espero que tenha compreendido bem todo este conteúdo e assimilado com os demais processos.

Até mais!

Referências

DAMAS, Luis. **Linguagem C**. 10. ed. Rio de Janeiro: Ltc, 2016.

MANZANO, José Augusto N G. **Linguagem C**: Acompanhada de uma xícara de café. São Paulo: Érica, 2015.

MIZRAHI, Victorine Viviane. **Treinamento em linguagem C**. São Paulo: Pearson Prentice Hall, 2008. 407 p.